

Лабораторная работа №1 "BMP конвертер"

Цель работы: В лабораторной работе предлагается изучить BMP формат. В качестве инструмента при выполнении лабораторной работы можно использовать любую систему программирования, позволяющую считывать и выводить файлы. Дается шаблон программы на языке C++.

Теоретическое обоснование

Формат BMP (от Bitmap), будучи одним из самых распространенных форматов хранения растровой графической информации, является стандартным для операционных систем Windows. В формате BMP изображение может храниться как без сжатия, так и со сжатием без потерь. Изображения могут быть монохромными (1 бит/пиксел) или цветными (4,8,16,24 или 32 бита/пиксел).

Файл в формате BMP состоит из четырех частей: BITMAPFILEHEADER, BITMAPINFOHEADER, RGBQUADS, Pixels.

BITMAPFILEHEADER
BITMAPINFOHEADER
RGBQUAD array
Color-index array

В BITMAPFILEHEADER и BITMAPINFOHEADER содержатся параметры файла и изображения, в RGBQUADS записывается цветовая палитра, а затем хранятся собственно пикселы изображения (как индексы палитры или как величины красной, зеленой и голубой составляющей цвета).

Формат структуры BITMAPFILEHEADER:

Название поля	Число байт	Комментарий
bfType	2	Тип файла. Должен быть BMP.
bfSize	4	Размер файла в байтах
bfReserved1	2	Зарезервировано. Должно быть 0
bfReserved2	2	Зарезервировано. Должно быть 0.
BfOffBits	4	Расстояние в байтах от BITMAPFILEHEADER до пикселей изображения

Формат структуры BITMAPINFOHEADER:

Название поля	Число байт	Комментарий
biSize	4	Размер структуры BITMAPINFOHEADER в байтах
biWidth	4	Ширина изображения в пикселах
biHeight	4	Высота изображения в пикселах
biPlanes	2	Число плоскостей на устройстве вывода. Должно быть 1
biBitCount	2	Число бит на пиксел (1,4,8,16,24,32)
biCompression	4	Метод хранения пикселей (BI_RGB, BI_RLE8, BI_RLE4, BI_BITFIELDS)
biSizeImage	4	Размер изображения в байтах (Может быть 0, если biCompression=BI_RGB)
biXPelsPerMeter	4	Горизонтальное разрешение устройства вывода (в пикселах/метр)
biYPelsPerMeter	4	Вертикальное разрешение устройства вывода (в пикселах/метр)
biClrUsed	4	Число цветовых индексов в таблице цветов, которые используются в изображении
biClrImportant	4	Число цветовых индексов, которые считаются важными при выводе изображения

Если величина biHeight положительна, то изображение записано снизу-вверх и начало изображения - левый нижний угол. Если величина biHeight отрицательна, то изображение записано сверху- вниз и начало изображения в левом нижнем углу.

Поле biCompression может принимать следующие значения:

BI_RGB (0) - формат без сжатия.

BI_RLE8 (1) - сжатие длинами серий, 8 бит/пиксел. Каждая запись состоит из 2х байтов, в первом байте хранится число цветовых индексов в серии, во втором байте цветовой индекс.

BI_RLE4 (2) - сжатие длинами серий, 4 бит/пиксел.

BI_BITFIELDS (3) - изображение хранится без сжатия, цветовая таблица состоит из трех четырехбайтовых масок для выделения красной, зеленой и голубой составляющей каждого пиксела. Этот режим используется при 16 и 32 битах/пиксел RGBQUADS и состоит из четверок байт rgbBlue, rgbGreen, rgbRed, rgbReserved, которые определяют голубую, зеленую и красную составляющую цвета. Размер массива RGBQUADS зависит от числа бит на пиксел и метода сжатия.

Значения поля **biBitCount**:

1 Изображение монохромное. RGBQUADS содержит две четверки, определяющие цветовые компоненты «черных» и «белых» пикселей. В этом случае каждый бит массива задает один пиксел.

4 Изображение содержит до 16 цветов, в RGBQUADS записано до 256 четверок, определяющих палитру изображения.

8 В изображении до 256 цветов. RGBQUADS содержит до 256 четверок, определяющих палитру изображения.

16 До 2^{16} цветов. Если `biCompression=BI_RGB`, то массив RGBQUADS пуст, на каждый пиксел изображения отводится 2 байта, в которых записаны B, G, R цветовые компоненты (5 бит/компоненту, старший бит двухбайтового слова не используется.) Если `biCompression=BI_BITFIELDS`, то RGBQUADS состоит из трех четырехбайтовых масок, определяющих R, G, B компоненты.

24 Если `biBitCount=24`, то массив RGBQUADS пуст и пиксели изображения хранятся в виде троек байт Blue, Green, Red.

32 Аналогично `biBitCount=16`, только на пиксел отводится 4 байта, три байта на R, G и B, старший байт не используется.

В настоящее время в файлах BMP изображения обычно хранятся без сжатия в формате либо 8 бит/пиксел (с палитрой) либо 24 бит/пиксел. Пиксели изображения хранятся в файле строка за строкой. Представление каждой строки должно быть выравнено на четырехбайтовую границу. Недостающие байты заполняются нулями.

Задание на выполнение работы

1. Используя пример, напишите программу для тестирования входного файла. Выведите размеры и число бит на пиксел (8-24 бита).

2. Напишите программу-конвертор для бинаризации BMP формата (24-1) с возможностью задания порога.

Пример программы конвертации 24-битного BMP изображения в 8-битное с палитрой серого:

```
#include <windows.h>
#include <iostream>
#include <string>

using namespace std;

int main()
{
```

```

// Вводимые и используемые переменные
string sInFileName, sOutFileName;
BITMAPFILEHEADER bmpFileHeader{}; // Заголовок файла входного BMP
BITMAPINFOHEADER bmpInfoHeader{}; // Заголовок информации входного BMP
int Width = 0, Height = 0;

RGBQUAD Palette[256]; // Палитра для выходного BMP (8 бит)
RGBTRIPLE* inBuf = nullptr; // Буфер строки входного BMP (3 байта на
пиксель: B,G,R)
BYTE* outBuf = nullptr; // Буфер строки выходного BMP (8-битные
индексы палитры)

HANDLE hInputFile = INVALID_HANDLE_VALUE; // Дескриптор входного файла
HANDLE hOutFile = INVALID_HANDLE_VALUE; // Дескриптор выходного файла
DWORD RW = 0; // Количество прочитанных/записанных
байт

// Ввод имени входного файла
cout << "Enter the full path of input BMP file: ";
cin >> sInFileName;

// Открываем входной файл для чтения
hInputFile = CreateFileA(
    sInFileName.c_str(),
    GENERIC_READ,
    FILE_SHARE_READ,
    NULL,
    OPEN_EXISTING,
    FILE_ATTRIBUTE_NORMAL,
    NULL
);

// Простейшая обработка ошибки: выходим если не удалось открыть
if (hInputFile == INVALID_HANDLE_VALUE) return 1;

// Ввод имени выходного файла
cout << "Enter the full path to output BMP file: ";
cin >> sOutFileName;

// Создаем выходной файл
hOutFile = CreateFileA(
    sOutFileName.c_str(),
    GENERIC_WRITE,
    0,
    NULL,
    CREATE_NEW,
    FILE_ATTRIBUTE_NORMAL,
    NULL
);

if (hOutFile == INVALID_HANDLE_VALUE) {
    CloseHandle(hInputFile);
    return 1;
}

// Чтение заголовков входного BMP
ReadFile(hInputFile, &bmpFileHeader, sizeof(bmpFileHeader), &RW, NULL);
ReadFile(hInputFile, &bmpInfoHeader, sizeof(bmpInfoHeader), &RW, NULL);

// Переходим к началу раstra входного BMP
SetFilePointer(hInputFile, bmpFileHeader.bfOfffBits, NULL, FILE_BEGIN);

```

```

// Извлекаем размеры изображения
Width = bmpInfoHeader.biWidth;
Height = bmpInfoHeader.biHeight;

// Выделяем буферы под одну строку входного изображения
inBuf = new RGBTRIPLE[Width];
outBuf = new BYTE[Width]; // выходная строка: индексы палитры (0..255)

// Подготавливаем заголовки выходного BMP (8 бит, палитра 256 цветов)
// bfOffBits: заголовок файла + заголовок информации + палитра
bmpFileHeader.bfOffBits = sizeof(bmpFileHeader)
    + sizeof(bmpInfoHeader)
    + 256 * sizeof(RGBQUAD);

bmpInfoHeader.biBitCount = 8; // 8 бит на пиксель в выходном
int rowSize = Width; // число пикселей в строке входного
int padding = (4 - (rowSize % 4)) % 4; // выравнивание по 4 байтам для строки в
BMP

// biSizeImage может быть нулевым, но здесь считаем реальную размерность
bmpFileHeader.bfSize = bmpFileHeader.bfOffBits + (rowSize + padding) * Height;

// Запись заголовков в выходной файл
WriteFile(hOutFile, &bmpFileHeader, sizeof(bmpFileHeader), &RW, NULL);
WriteFile(hOutFile, &bmpInfoHeader, sizeof(bmpInfoHeader), &RW, NULL);

// Заполнение палитры для 8-битного BMP: градации серого
for (int i = 0; i < 256; ++i) {
    Palette[i].rgbBlue = static_cast<BYTE>(i);
    Palette[i].rgbGreen = static_cast<BYTE>(i);
    Palette[i].rgbRed = static_cast<BYTE>(i);
    Palette[i].rgbReserved = 0;
}

// Запись палитры
WriteFile(hOutFile, Palette, sizeof(Palette), &RW, NULL);

// Обработка строк изображения: конвертация в оттенки серого
for (int y = 0; y < Height; ++y) {
    // Считываем одну строку 24-битного входного BMP: Width пикселей по 3 байта
    ReadFile(hInputFile, inBuf, sizeof(RGBTRIPLE) * Width, &RW, NULL);

    for (int x = 0; x < Width; ++x) {
        // Преобразование по формуле восприятия серого
        BYTE gray = static_cast<BYTE>(
            0.3f * inBuf[x].rgbtRed +
            0.59f * inBuf[x].rgbtGreen +
            0.11f * inBuf[x].rgbtBlue
        );
        outBuf[x] = gray;
    }

    // Запись обработанной строки в выходной файл
    WriteFile(hOutFile, outBuf, sizeof(BYTE) * Width, &RW, NULL);

    // Выравнивание строки (padding) для выходного 8-битного BMP
    if (padding) {
        BYTE pad[3] = {0, 0, 0};
        WriteFile(hOutFile, pad, padding, &RW, NULL);
    }

    // 16) Пропуск выравнивания во входном файле, если оно есть

```

```

        int inPadding = (4 - ((Width * 3) % 4)) % 4;
        if (inPadding) SetFilePointer(hInputFile, inPadding, NULL, FILE_CURRENT);
    }

    // Очистка ресурсов
    delete[] inBuf;
    delete[] outBuf;
    CloseHandle(hInputFile);
    CloseHandle(hOutFile);

    cout << "Conversion completed successfully!" << endl;
    system("pause");
    return 0;
}

```

Содержание отчета

1. Цель работы.
2. Задание на работу.
3. Исходное изображение.
4. Исходный код и скриншот вывода для первого задания.
5. Исходный код и выходное изображение с указанием порога бинаризации.
6. Выводы по работе.